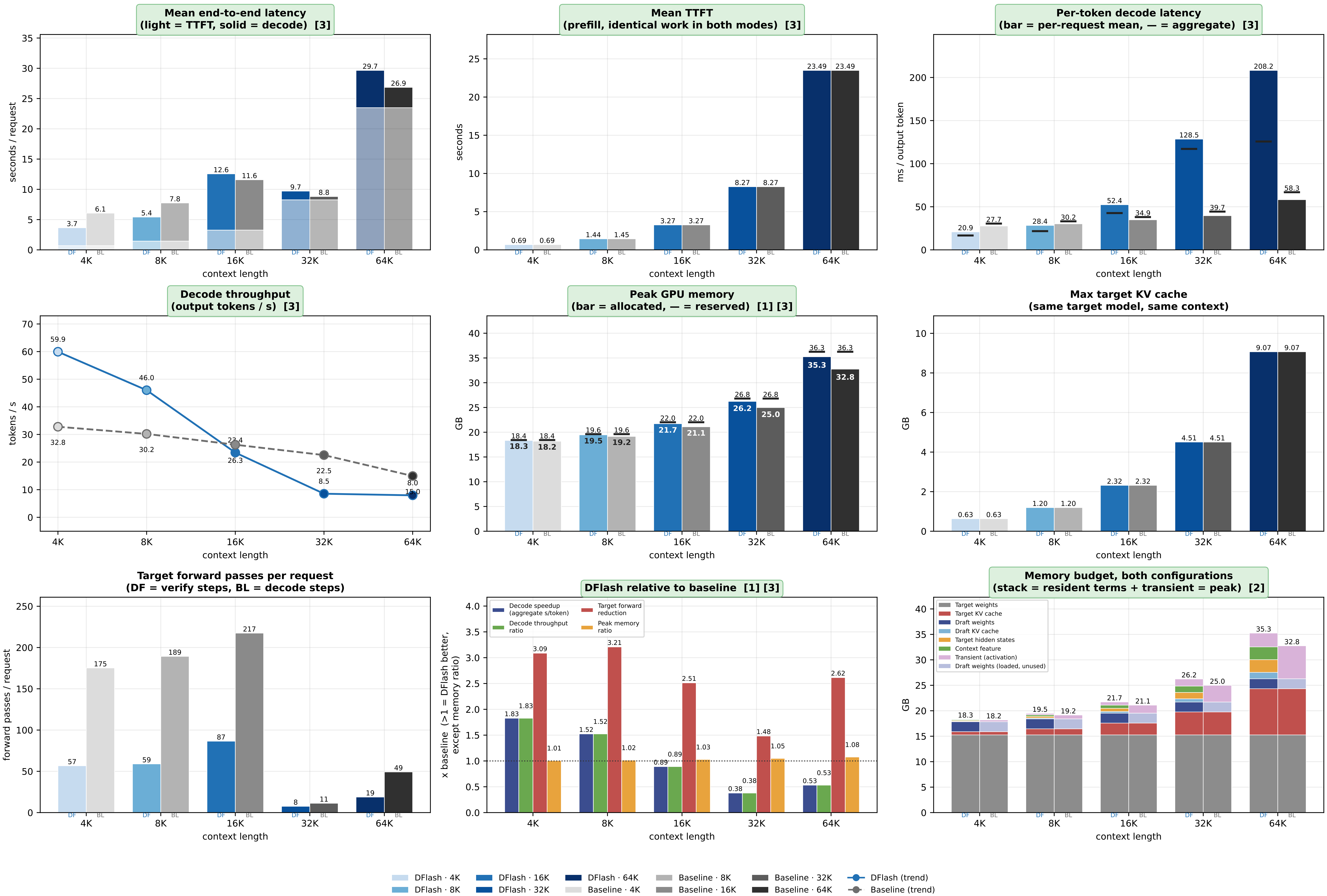
DFlash vs. baseline — qwen3-8b on LongBench-E, benchmark-wide averages, hidden_states=selective (4K / 8K / 16K / 32K / 64K; draft-only metrics excluded)



Samples per context length (qwen3-8b) — 4K: n=32 (13 datasets) · 8K: n=32 (12 datasets) · 16K: n=32 (10 datasets) · 32K: n=32 (9 datasets, 31 composed) · 64K: n=32 (8 datasets, 32 composed)
Mean output tokens per request [DFlash / baseline] — 4K: 178 / 176 · 8K: 183 / 190 · 16K: 218 / 218 · 32K: 12 / 12 · 64K: 49 / 50 (long-context runs emit far fewer tokens; compare per-token metrics, not per-request ones)

Panels highlighted in green differ from visualization/ (hidden_states=full) – ADDED = a quantity that did not exist in those records, REVISED = same quantity, changed measurement

[1] REVISED hidden_states=selective. The drafter reads the target's residual streams through forward hooks on the injected layers rather than output_hidden_states=True, so the tuple the target materialises drops from L_t+1 layers to n_inj. Only DFlash pays that term, so only the DFlash bars move; the baseline's peak is unchanged from the full-mode records. Acceptance and output tokens are identical in both modes.

[2] ADDED summary.*.memory_budget and target_weight_gb, new record fields, for both configurations. The baseline panel lists the draft weights it never calls, because the drafter is resident on the same card for the whole benchmark -- which is why the peak delta between the two bars is not the drafter's cost.

[3] REVISED Device count differs from the full-mode records: 8B/64K 2->1. Selective frees enough memory to fit on fewer cards. No run here is sharded any more, so every column is single-GPU. Memory stays comparable across a device change; timings do not -- going from sharded to single-GPU speeds the *baseline* up more than it speeds DFlash, so a speedup can read lower without DFlash having got slower.